Suche mit Branch-and-Bound

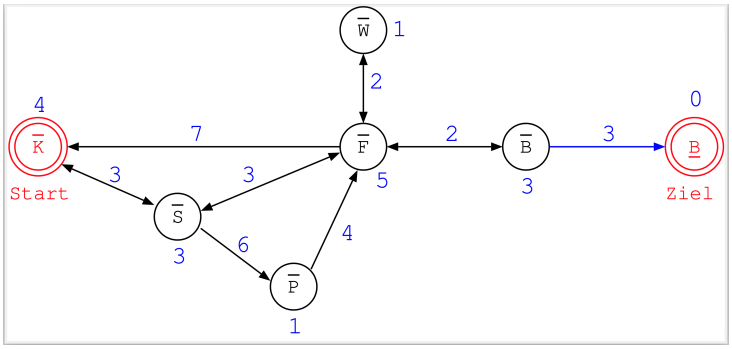
Carsten Gips (HSBI)

TL;DR
Branch-and-Bound gehört zu den “Informierten Suchverfahren”: Es werden (reale) Pfadkosten statt der Anzahl der Schritte berücksichtigt.
Branch-and-Bound entsteht, wenn man bei der Tree-Search oder der Graph-Search für die Datenstruktur eine sortierte **Queue** (Prioritätsqueue) benutzt: Expandierte Nachfolger werden immer **hinten** in die Queue eingefügt, diese wird nach den **Kosten der partiellen Pfade sortiert** und der nächste zu expandierende Knoten (d.h. der bisher günstigste partielle Weg) wird **vorn** aus der Queue genommen. Branch-and-Bound arbeitet mit den bisher entstandenen (realen) Kosten der partiellen Wege.
Videos

- VL Branch-and-Bound

---

1 Hole das Buch



=> **Problemlösen == Suche im Graphen**

Informierte Suche: Nutzung der Kostenfunktion:

Gesamtkosten: $f(n) = g(n) + h(n)$

- $n \in S$ auf aktuellem Weg erreichter Knoten
- $g(n)$ tatsächliche Kosten für Weg vom Start bis Knoten n
- $h(n)$ geschätzte Restkosten für Weg von Knoten n zum Ziel => $h(n)$ wird auch “heuristische Funktion” oder “Heuristik” genannt

Varianten:

- **Branch-and-Bound**
- Best First
- A*

2 Branch-and-Bound (*BnB*)

Variante der Breitensuche mit Kosten

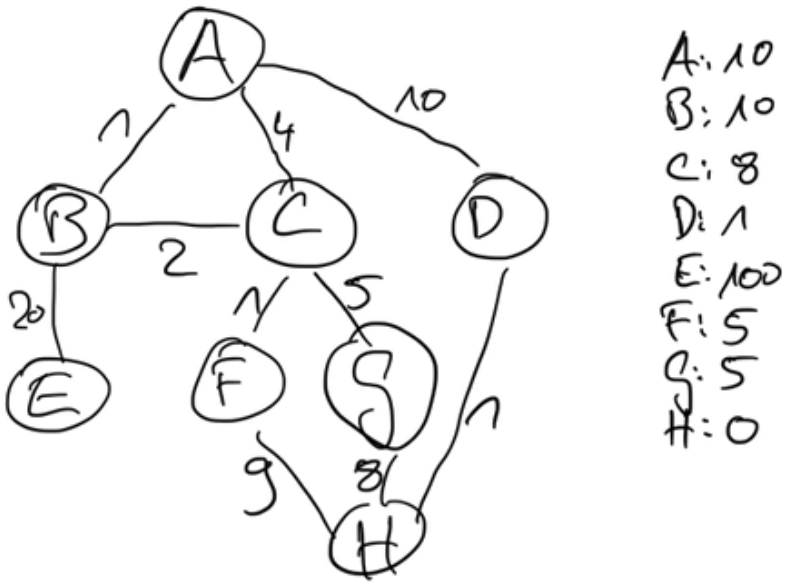
- Idee: Expandiere den *bisher* günstigsten partiellen Weg

- Kostenfunktion: $f(n) = g(n)$
- Datenstruktur: **sortierte Queue** (Prioritätsqueue)

- Voraussetzung: alle Aktionen haben positive Kosten (jeden Knoten n gilt: $g(n) > 0$)

Hinweis: Die Branch-and-Bound-Suche taucht im (Russell und Norvig 2020) als Erweiterung der "Uniformen Suche" auf …

3 BnB: Finde einen Weg von A nach H



4 Bemerkungen zu BnB mit Graph-Search

Graph-Search fordert: Expandierte Nachfolgerknoten, die schon in der Queue sind, sollen nicht (erneut) in die Queue aufgenommen werden.

- Problem dabei: Was ist mit den Kosten?! Der neu expandierte Weg könnte günstiger sein als der schon in der Queue enthaltene.

- Lösung (vgl. Optimierungsmöglichkeiten für A*):

  Füge zunächst alle neu expandierten partiellen Pfade (mit unmarkierten Endknoten) in die Queue ein, sortiere diese und behalte von mehreren Pfaden zum gleichen Knoten nur den jeweils günstigsten in der Queue

Pfade, deren Endknoten bereits früher im Pfad vorkommt (Schleifen), werden bei Graph-Search in Schritt 2 nicht in die Queue aufgenommen (der Endknoten wäre bei einer Schleife ja bereits markiert und der neue Pfad würde bei Graph-Search nicht weiter beachtet).

Das Einfärben ist kein Problem, da nur der jeweils günstigste Knoten aus der Queue genommen, gefärbt und expandiert wird. D.h. alle anderen Wege sind zu diesem Zeitpunkt bereits teurer. Wenn man nun (später) über einen anderen Weg zu einem bereits gefärbten Knoten kommt, kann der neue Weg nicht günstiger sein (positive Kosten vorausgesetzt).

5 Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2020)). Dies erschwert die Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Auf dem Papier sortiert sich die Queue schlecht, deshalb können Sie darauf verzichten, wenn Sie den im nächsten Schritt zu expandierenden Weg unterstreichen. Wer nicht mit Unterstreichen arbeiten will, muss eben dann manuell sortieren …

Wenn bei der Graph-Search-Variante ein Weg nicht in die Queue aufgenommen wird, weil bereits ein anderer (günstigerer) Weg zum selben (Zwischen-/End-) Knoten bereits in der Queue enthalten ist, schreiben Sie dies geeignet auf. Dies gilt auch für den analogen Fall, wenn ein Weg aus der Queue entfernt wird, weil ein günstigerer Weg zum selben (Zwischen-/End-) Knoten eingefügt werden soll.

6 Eigenschaften von BnB

Siehe A*

7 Wrap-Up

- Informierte Suchverfahren
  - Nutzen reale Pfadkosten und/oder Schätzungen der Restkosten
  - Branch-and-Bound: nur reale Pfadkosten $g(n)$

8 Zum Nachlesen

- Russell und Norvig (2020): Branch-and-Bound (Uniforme Suche): Abschnitt 3.4.2

Lernziele

- k2: Verwendete Datenstrukturen
- k2: Algorithmische Abläufe, Terminierung
- k2: Optimalität, Vollständigkeit und Komplexität
- k2: Bedeutung nicht-negativer Kosten für Branch-and-Bound
- k3: Informierte Suchverfahren: Branch-and-Bound

Quizzes

- Selbsttest Branch-and-Bound (ILIAS)

Quellen

Russell, S., und P. Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 5f0b0a3 (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-17)